Handwritten Digit Recognition

using K-Nearest Neighbors

Machine Learning Project Report

Author: Aryan Budukh

Date: March 2026

MNIST Dataset | scikit-learn | FastAPI | GitHub Pages

Project Overview

Objective

Build a handwritten digit recognition system using K-Nearest Neighbors (KNN) on the MNIST dataset, with a web-based interactive demo.

Dataset: MNIST

70,000 grayscale images of handwritten digits (0-9), each 28x28 pixels.

Training set: 60,000 samples | Test set: 10,000 samples.

One of the most well-studied benchmarks in machine learning.

Approach

1. Deskewing preprocessing: moment-based affine transformation.
2. PCA dimensionality reduction: 784 -> 50 principal components.
3. Distance-weighted KNN classification with K=3.

Tech Stack

Python, scikit-learn, NumPy, matplotlib, FastAPI,

HTML5 Canvas, CSS3, JavaScript, GitHub Pages, Railway.

Preprocessing Pipeline

1. Deskewing

Moment-based affine transformation to straighten slanted digits.

Computes the second-order central moments of the image to determine the skew direction, then applies an affine warp to correct it.

Impact: Reduces error rate from ~3.1% to ~1.69% (a 45% reduction).

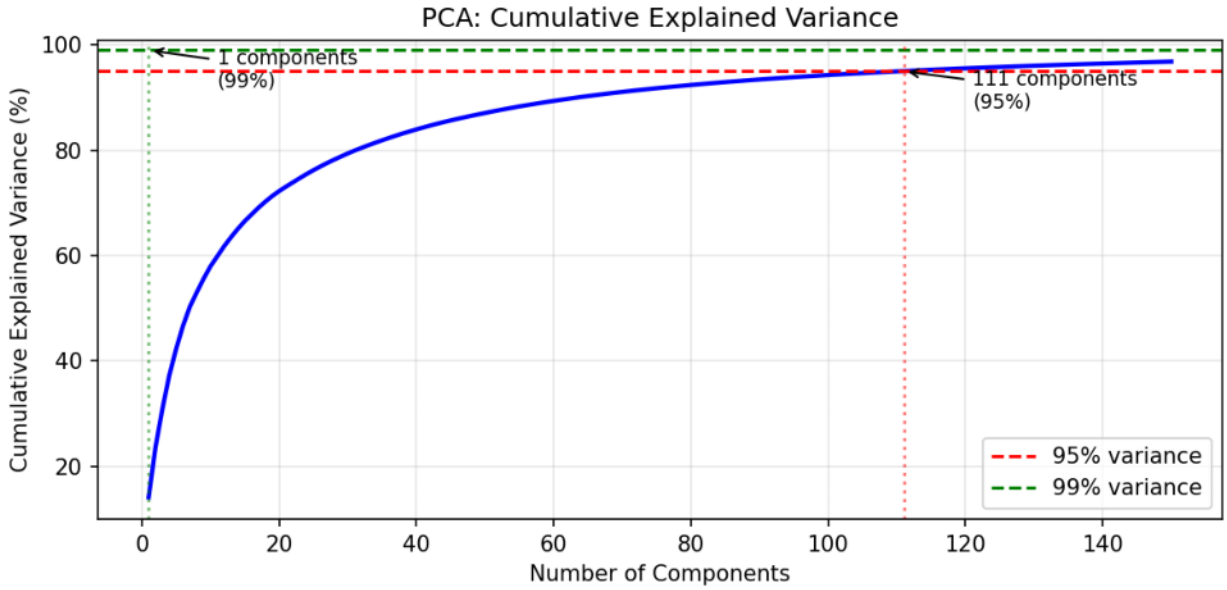
2. PCA Dimensionality Reduction

Reduces 784-dimensional pixel space to 50 principal components.

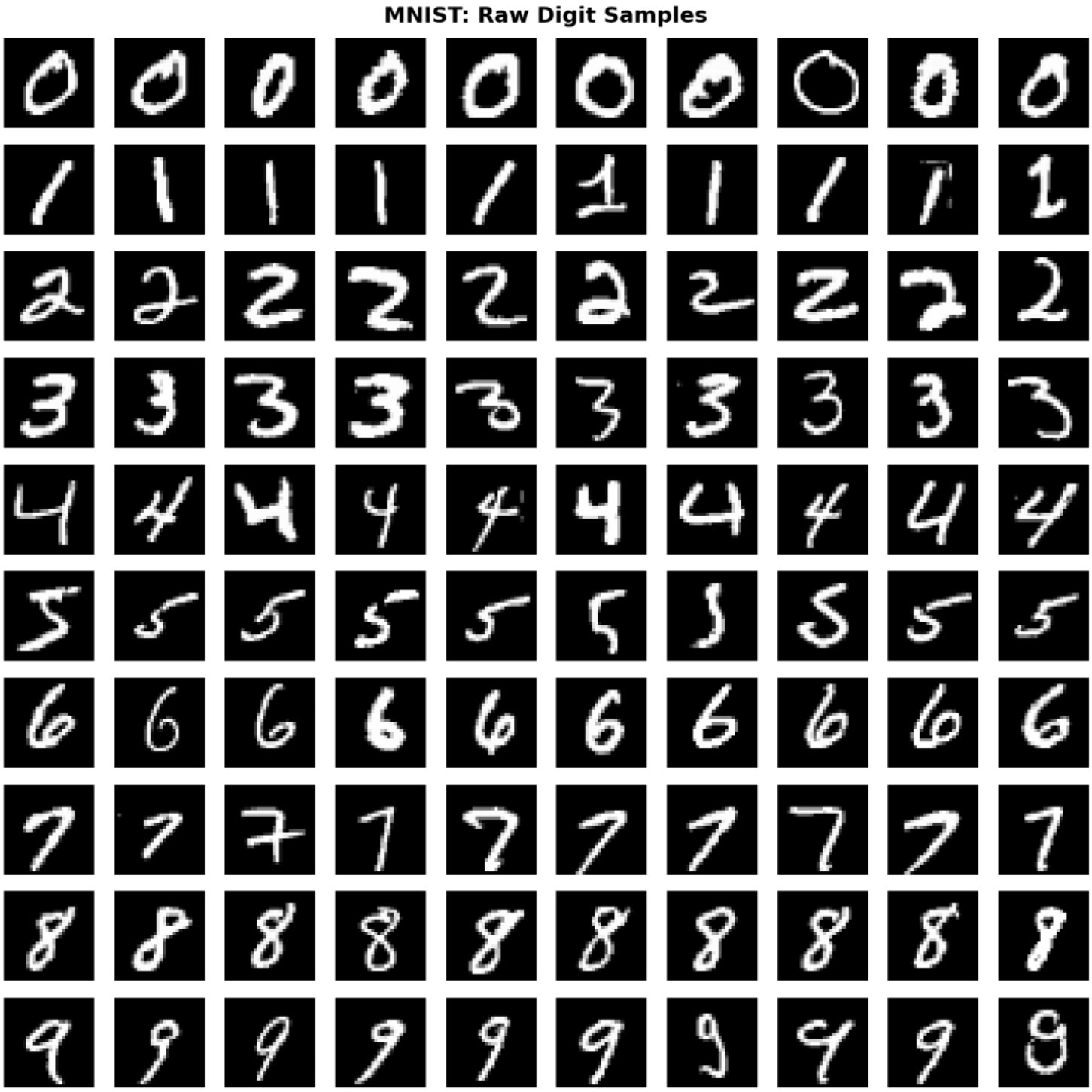
Retains 87.1% of the total variance.

Benefits: Faster KNN inference, reduced noise, better generalization.

PCA Explained Variance vs Number of Components



Sample Digits

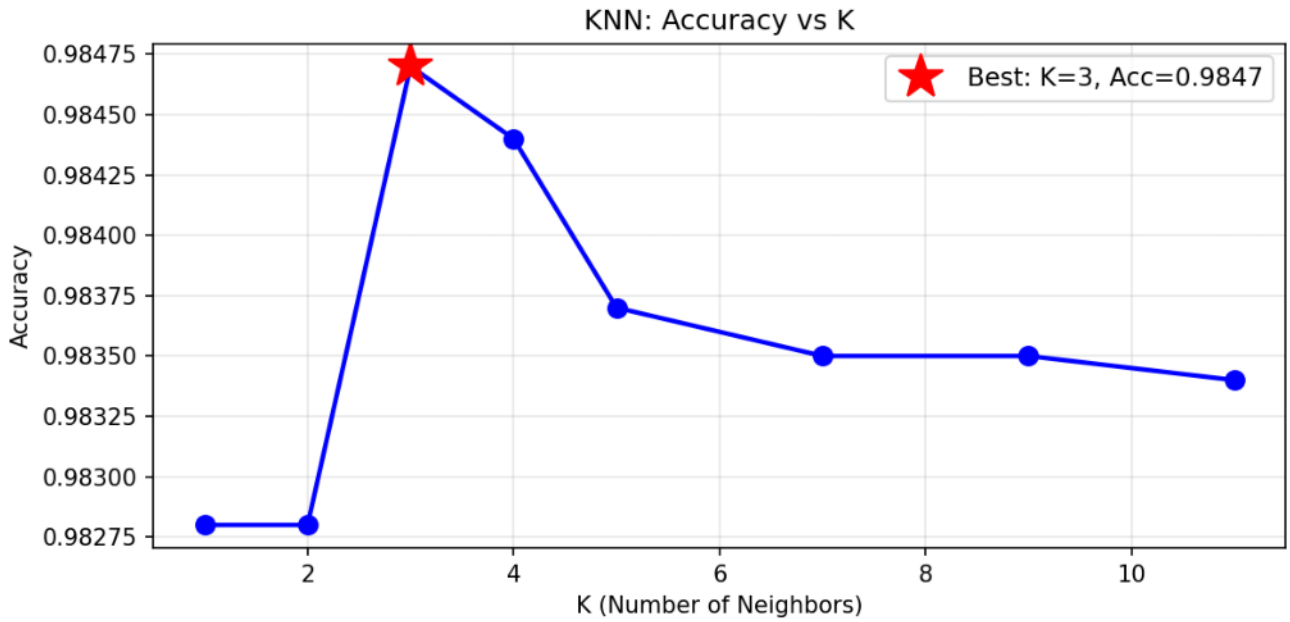


Sample digits from MNIST dataset -- 10 examples per class

Deskewing Comparison

Effect of deskewing preprocessing -- before (top) vs after (bottom)

Hyperparameter Optimization



K	Accuracy	
1	98.28%	
2	98.28%	
3	98.47%	BEST
4	98.44%	
5	98.37%	
7	98.35%	
9	98.35%	
11	98.34%	

Model Configuration

Parameter	Value
Algorithm	K-Nearest Neighbors
K	3 (optimal)
Distance Metric	Minkowski (Euclidean, p=2)
Weights	Distance-weighted
PCA Components	50
Preprocessing	Deskewing + PCA
Cross-Validation	5-fold, 97.93% +/- 0.26%

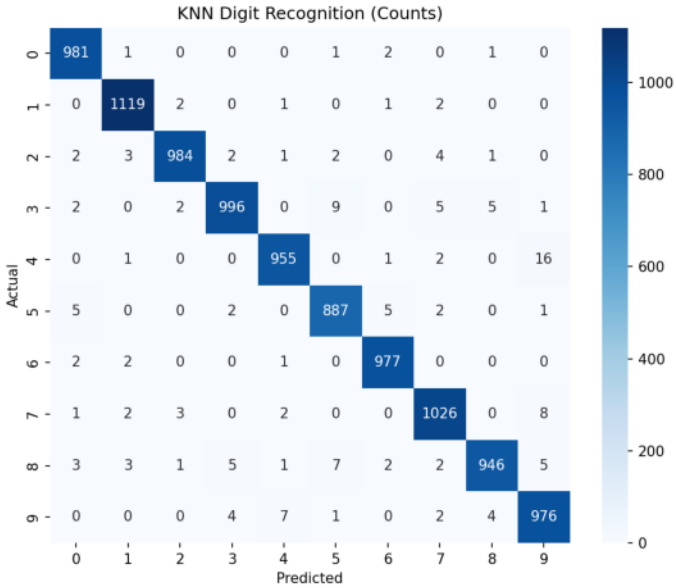
Distance-weighted KNN gives closer neighbors more influence, which improves accuracy compared to uniform weighting.

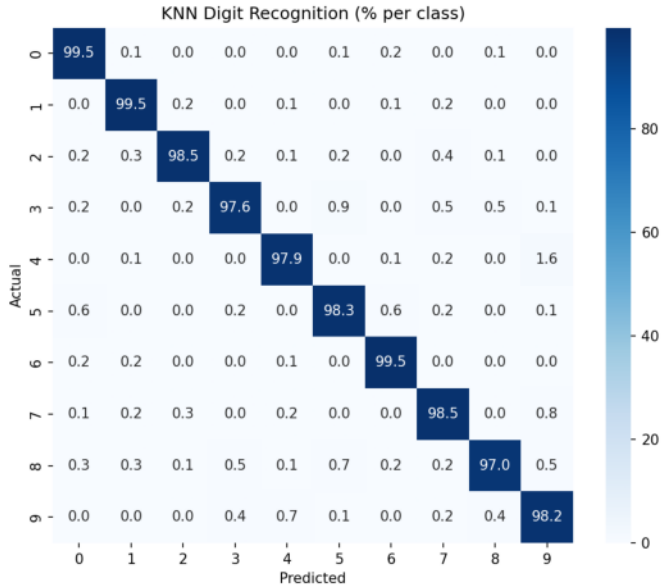
Results

98.47%

Test Accuracy

Error Rate: 1.53% | 153 misclassified out of 10,000 test samples





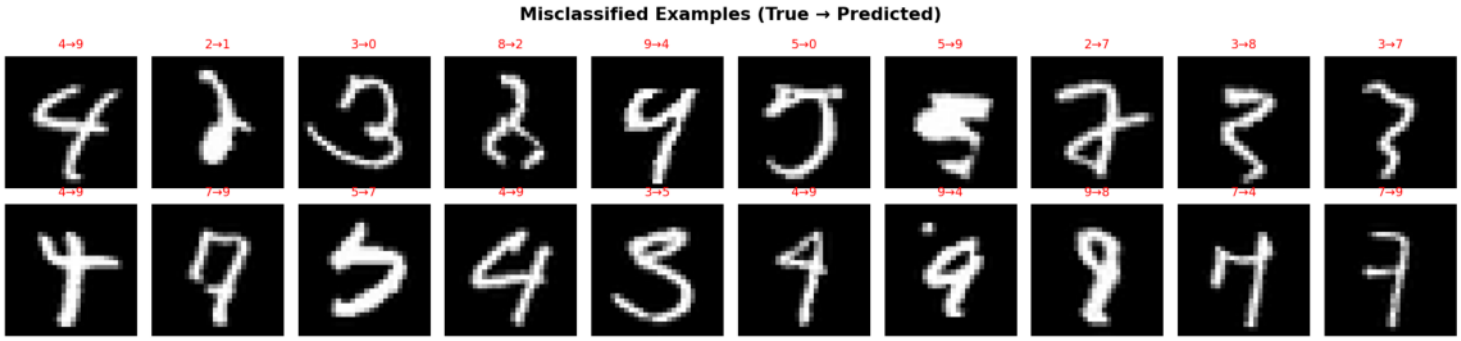
Per-Class Performance

Digit	Precision	Recall	F1-Score	Support
0	0.9849	0.9949	0.9899	986
1	0.9894	0.9947	0.9920	1125
2	0.9919	0.9850	0.9884	999
3	0.9871	0.9765	0.9818	1020
4	0.9866	0.9795	0.9830	975
5	0.9779	0.9834	0.9807	902
6	0.9889	0.9949	0.9919	982
7	0.9818	0.9846	0.9832	1042
8	0.9885	0.9703	0.9793	975
9	0.9692	0.9819	0.9755	994

Best F1-Score: Digit 1 (0.9920) | Worst F1-Score: Digit 9 (0.9755)

All classes achieve F1-Score above 0.97 -- strong balanced performance.

Misclassified Examples

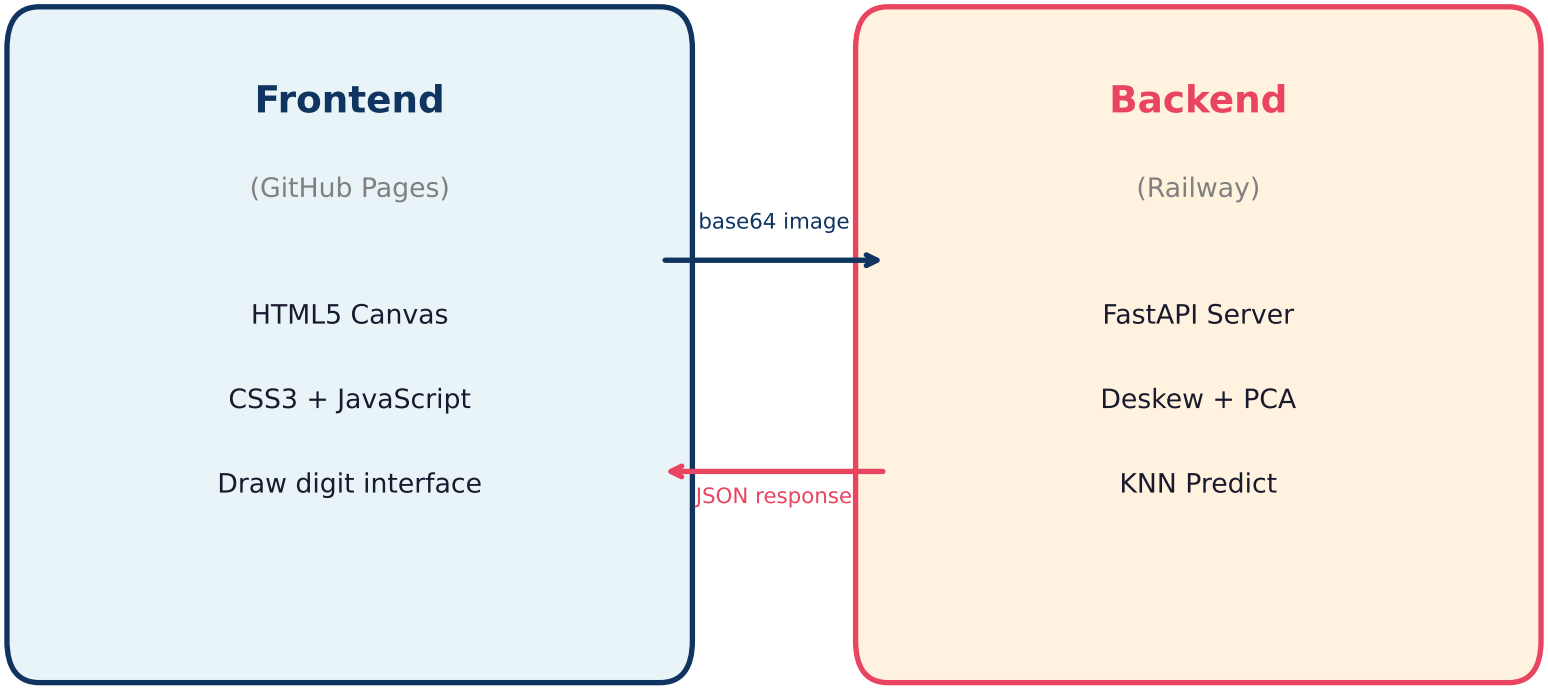


Analysis of Misclassifications

Most errors occur between visually similar digits: 4<->9, 3<->8, 7<->9.

These are cases where even humans might struggle with ambiguous handwriting.

System Architecture



Request Pipeline

1. User draws digit on HTML5 Canvas
2. Canvas image encoded as base64 PNG
3. POST request to FastAPI /predict endpoint
4. Backend: decode image -> deskew -> PCA transform
5. KNN classifies the preprocessed image
6. Predicted digit + confidence returned as JSON
7. Frontend displays result with confidence bar

Deployment

Frontend: GitHub Pages

URL: https://skullfire07.github.io/ML-FINAL/

Static site served via GitHub Pages.

HTML5 Canvas for digit drawing, JavaScript for API calls.

Backend: Railway

URL: https://digit-recognition-production.up.railway.app

FastAPI application with automatic deployment from GitHub.

Includes /predict endpoint for digit classification.

Model artifacts (KNN, PCA, scaler) loaded at startup.

Source Code: GitHub

Repository: https://github.com/SKULLFIRE07/ML-FINAL

Includes training scripts, API server, frontend, and documentation.

Key Files

train_model.py -- Model training and evaluation pipeline

main.py -- FastAPI backend server

index.html -- Frontend web application

knn_model.pkl -- Trained KNN model artifact

pca_model.pkl -- Fitted PCA transformer

Conclusion

Key Achievements

Achieved 98.47% accuracy using KNN on MNIST -- competitive with simple neural networks.

Built a full end-to-end pipeline: training, evaluation, API, and web frontend.

Deployed as a live interactive demo accessible via browser.

Key Insights

Deskewing is the single most impactful preprocessing step (45% error reduction).

PCA not only speeds up inference but also improves accuracy slightly via denoising.

KNN with proper preprocessing is competitive with simple neural networks on MNIST.

Distance-weighted voting outperforms uniform voting consistently.

Future Work

Tangent distance metric: Could achieve ~98.9% accuracy by accounting for small transformations (rotation, scaling, thickness) in distance computation.

Shape context features: Could push accuracy to ~99.4% by using richer structural descriptors instead of raw pixel values.

Ensemble methods: Combining KNN with other classifiers for even better results.

References

[1] MNIST Database

Yann LeCun, Corinna Cortes, Christopher J.C. Burges.

http://yann.lecun.com/exdb/mnist/

[2] Shape Matching and Object Recognition Using Shape Contexts

Serge Belongie, Jitendra Malik, Jan Puzicha (2002).

IEEE Transactions on Pattern Analysis and Machine Intelligence.

[3] Modified Sliding-Window Metric for KNN

arXiv:1809.06846.

https://arxiv.org/abs/1809.06846

[4] scikit-learn: Machine Learning in Python

Pedregosa et al., JMLR 12, pp. 2825-2830, 2011.

https://scikit-learn.org/

[5] FastAPI Documentation

Sebastian Ramirez.

https://fastapi.tiangolo.com/